

Real-Time Visualization of Digital Modulation and Channel Impairments Using a Single PlutoSDR

Howard Wang

ENGR 78 – Final Project

Spring 2026

The Problem

“I can solve the BER equation, but I have no idea what a noisy constellation *looks* like.”

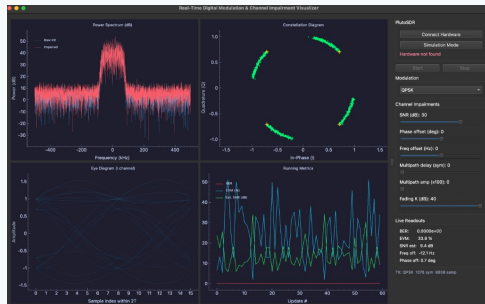
- Traditional courses: modulation = equations + static textbook plots
- Students memorize $Q(\sqrt{2E_b/N_0})$ but never **watch** a signal degrade
- Gap between theory and physical intuition

What if you could see every impairment happen in real time, on real hardware?

What We Built

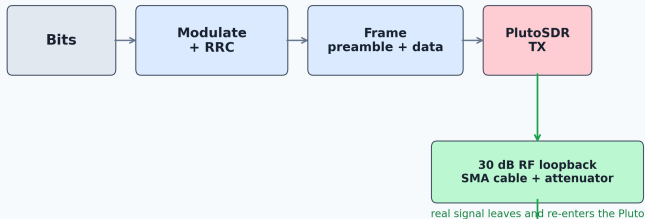
A **real-time, interactive** digital-communications workbench:

- 1 Generate & transmit a digital signal through an **ADALM-PLUTO SDR**
- 2 Receive it back via loopback (or software simulation)
- 3 Inject impairments with sliders — **instantly** see the effect
- 4 Five live panels: spectrum, constellation, eye diagram, metrics, **throughput**
- 5 Automated **BER sweep mode** for quantitative analysis

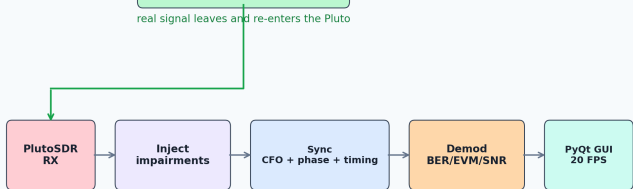


System Architecture

Transmit chain on host PC



Receive + DSP chain



How to Read the Architecture

Transmit Side

- **Payload bits:** random data to test the link.
- **Symbol map + RRC:** turns bits into constellation points and limits bandwidth.
- **Frame builder:** adds preamble and payload so RX can lock and compare bits.

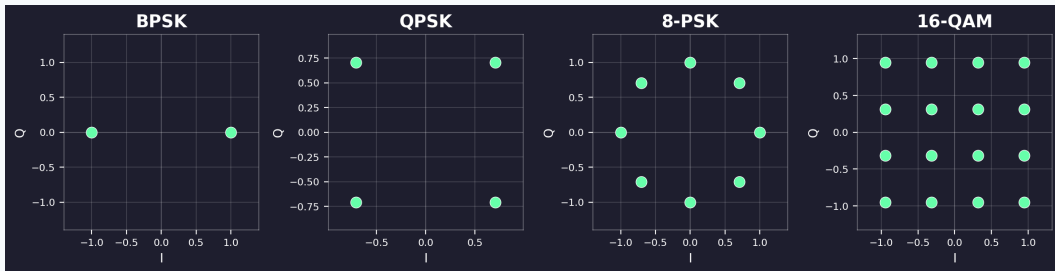
Receive Side

- **Pluto loopback:** RF leaves TX, passes through attenuation, then re-enters RX.
- **Software impairments:** adds noise, phase/frequency offset, fading, and multipath after RX.
- **Sync + demod:** fixes timing/phase/frequency, recovers bits, and reports BER/EVM/SNR.

Why the impairment block is after RX

Keeping impairments in software makes the sliders instant and lets hardware loopback and simulation use the same DSP + GUI path.

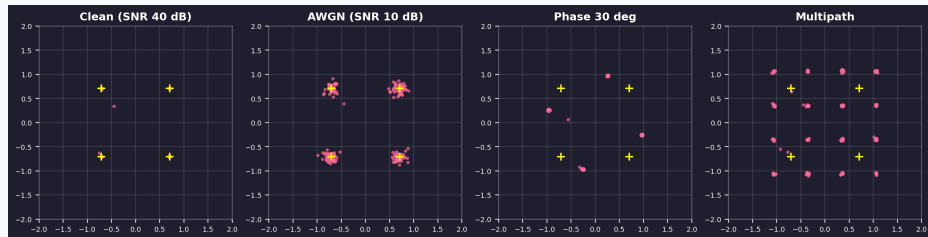
Four Modulation Schemes



- Gray-coded constellations
- Root-raised-cosine pulse shaping ($\alpha=0.35$, 8 samp/sym)

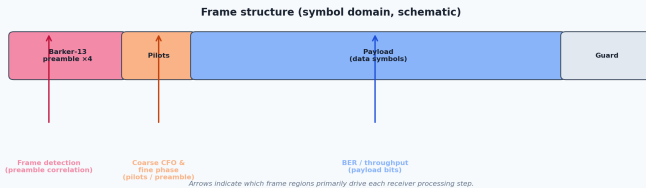
- Nearest-neighbor hard-decision demod
- Switchable on-the-fly from the GUI

Channel Impairments – See the Damage



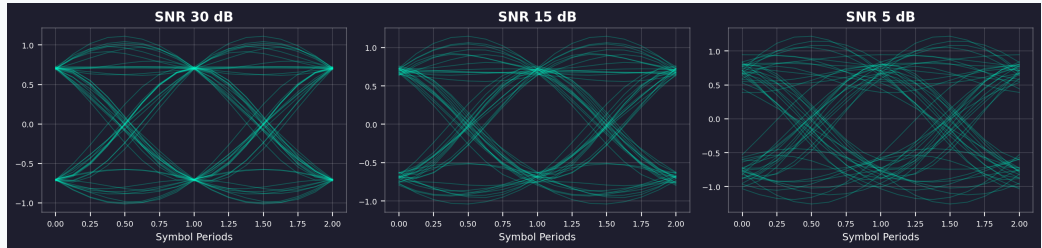
Impairment	Model	Control Range
AWGN	Complex Gaussian	0–40 dB SNR
Phase offset	$e^{j\phi}$ rotation	$\pm 180^\circ$
Frequency offset	Linear phase ramp	± 5000 Hz
Multipath	2-tap delay line	0–10 sym, gain 0–1
Flat fading	Rician envelope	$K = -10$ to 40 dB

Receiver Synchronization



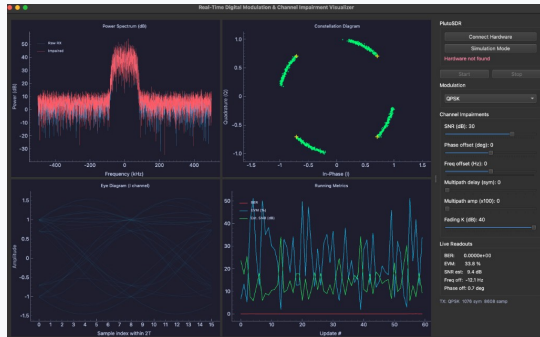
- 1 **Frame detection** – cross-correlate with Barker-13 template
- 2 **Freq offset est.** – repeated-preamble phase difference
- 3 **Phase est.** – pilot-aided from known preamble
- 4 **Timing recovery** – Gardner TED with first-order loop

Eye Diagram – Reading Signal Health



- 50 overlaid symbol-period traces of the matched-filtered I channel
- **Wide-open eye** $\Rightarrow$ clean decisions
- **Closed eye** $\Rightarrow$ errors imminent
- Instantly shows ISI from multipath, jitter from timing error, or noise

Live Mode – GUI Walkthrough



Spectrum
FFT of IQ
spectral shape

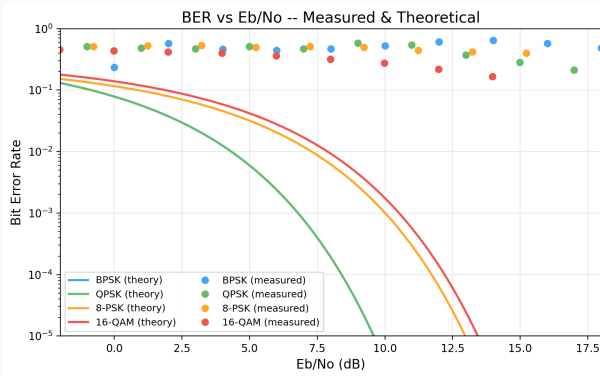
Constellation
demod symbols
ideal refs

Eye Diagram
ISI indicator
timing quality

Metrics
rolling BER,
EVM, SNR

+ **Throughput & Cumulative Bits** panel across the bottom row

BER Sweep Mode – Theory Meets Measurement



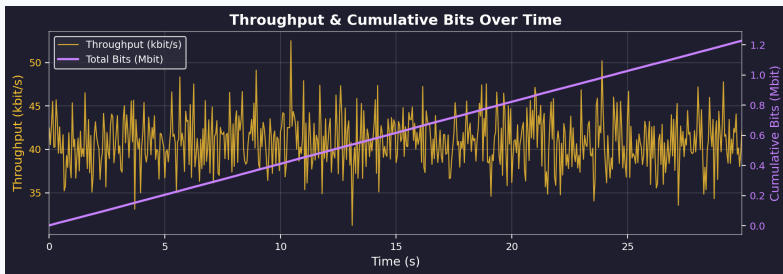
- Automated: sweep SNR, measure BER for each scheme
- Circles = measured, dashed = theory:

$$\text{BPSK/QPSK: } Q\left(\sqrt{2E_b/N_0}\right)$$

$$\text{16-QAM: } \frac{3}{8} \operatorname{erfc}\left(\sqrt{0.4E_b/N_0}\right)$$

- **Validates** that our pipeline matches textbook curves
- Hardware loopback adds ~0.5–1 dB loss vs. simulation

Throughput & Cumulative Bits



Instantaneous Throughput

- QPSK at 20 FPS: ~ 40 kbit/s decoded payload
- 16-QAM doubles it: ~ 80 kbit/s
- Varies with USB latency and sync lock

Cumulative Bits

- Tracks **every bit** decoded since “Start”
- Linear ramp = steady link; plateau = sync loss
- > 1 Mbit within 30 s of clean QPSK

Software Architecture

Module	Role	Key Technique
modulation.py	Constellation, mod/demod, RRC	Gray-coded maps
impairments.py	AWGN, phase, freq, multipath	Chained transforms
synchronization.py	Preamble, freq/phase/timing	Gardner TED
pluto_interface.py	PlutoSDR TX/RX	USB auto-detect
dsp_worker.py	QThread pipeline	Shared-data + mutex
ber_sweep.py	BER sweep engine	SNR loop + theory
gui.py	PyQt6 + PyQtGraph	20 FPS timer poll

Key Design Choice

DSP worker writes to shared memory → GUI timer reads at its own pace.

Result: **no signal-queue backlog**, consistent frame rate regardless of DSP speed.

Demo Scenarios – Try It Yourself

Noise Floor Exploration

- Start with all impairments off
- Slide SNR from 40 dB $\rightarrow$ 0 dB
- Watch: constellation spreads, eye closes, BER climbs

Multipath

- Delay = 3 symbols, gain = 0.5
- Eye diagram shows ISI directly
- Clusters blur and merge

Phase & Frequency Offset

- 45° phase: constellation rotates
- 2 kHz freq offset: symbols spin (sync corrects it)

Scheme Comparison

- Fix SNR at 15 dB, switch BPSK $\rightarrow$ 16-QAM
- More bits/symbol = less noise margin
- BER sweep quantifies the trade-off

Results & Observations

Quantitative

- Measured BER curves match theory within < 1 dB for all four schemes
- Hardware loopback adds ~ 0.5 – 1 dB loss (I/Q imbalance, DC offset, quantization)
- Gardner TED locks timing within 2–3 symbol periods

Performance

- Simulation mode: ~ 20 FPS, 6 ms DSP/frame
- Hardware mode: ~ 15 FPS (USB latency)
- Sustained ~ 40 kbit/s (QPSK) to ~ 80 kbit/s (16-QAM) decoded throughput
- > 1 Mbit cumulative within 30 s

Lessons Learned

- ❶ **Synchronization is the hard part** – modulation is textbook, but preamble detection + timing recovery required iterative tuning
- ❷ **Decouple DSP from GUI** – direct signal emissions caused Qt event-queue backlog; shared memory + timer polling fixed it
- ❸ **Hardware surprises** – real PlutoSDR has DC offset, I/Q imbalance, and USB latency that pure math ignores
- ❹ **Visualization builds intuition** – watching the constellation rotate under frequency offset taught me more than any derivation

Conclusion

Built a [complete real-time digital communications visualizer](#) with:

- ✓ 4 modulation schemes (BPSK, QPSK, 8-PSK, 16-QAM)
- ✓ 6 controllable channel impairments with instant visual feedback
- ✓ Full receiver synchronization chain
- ✓ Live throughput & cumulative bits tracking
- ✓ Two modes: interactive live exploration & automated BER sweep
- ✓ Hardware or simulation – works everywhere

All open-source Python – ~1200 lines across 7 modules.

Thank You – Questions?